A Proposal: An Anonymous Identification Verification System

Based on Ring Signatures with Isolated Smart Card Hardware

Dinh Anh Khoa Nguyen - 245849840 - nguy9840@mylaurier.ca
Jiangtao Wang - 245823800 - Wang2370@mylaurier.ca
Leila Chen - 245850620 - chen5062@mylaurier.ca
Yuyang Wu - 245817310 - wuxx7310@mylaurier.ca

January 22, 2026

Abstract

This proposal outlines the engineering implementation of a privacy-preserving messaging platform that decouples message integrity from identity revelation. Leveraging Yubikey hardware PRF and nested Ring Signatures, the system allows users to send undeniable, tamper-proof messages anonymously, with the exclusive capability to retroactively prove authorship. The project adopts an Agile methodology, utilizing React/TypeScript for the frontend and Node.js for the backend.

Contents

1 Executive Summary and Highlights **3**
1.1 Overview . . . 3
1.2 Key Highlights . . . 3

2 Epic **4**
2.1 What is the problem? . . . 4
2.2 What is our goals? . . . 4

3 Story **4**
3.1 How do we provide the solution? . . . 4

4 Technical Model **5**
4.1 Security Model . . . 5
4.2 Technical Stack Chosen . . . 5
4.3 Cryptographic Construction . . . 5

5 System Design **6**
5.1 Architecture Overview . . . 6
5.2 Component Diagram . . . 6
5.3 Data Flow . . . 6

6 Genesis Sprint and Milestones **6**
6.1 Sprint 0: Feasibility & PoC (2 Weeks) . . . 6
6.2 Milestones 1: Core Cryptography Module . . . 6
6.3 Milestones 2: Backend & Basic UI . . . 7
6.4 Milestones 3: Integration & The "Reveal" Flow . . . 7

1 Executive Summary and Highlights

1.1 Overview

Project AIVS (Anonymous Identification Verification System) is a cryptographic engineering initiative designed to solve the a paradox in secure communications: the trade-off between anonymity and accountability, and introducing a hardware smartcard to participate in this process to ensure the security of the private keys.

1.2 Key Highlights

- **Hardware Security:** Utilizes Yubikey PRF to generate unpredictable passphrases, ensuring keys never leave the hardware boundary.
- **Nested Ring Signatures:** Employs a dual-layer signing mechanism (Inner Ring for identity proof, Outer Ring for anonymous transmission) to ensure mathematical unlinkability.
- **Selective Disclosure:** Grants the sender full control over identity revelation. No central authority can unmask the user; only the holder of the physical hardware token can decrypt the proof.
- **Agile Engineering Stack:** Built on a modern, type-safe stack (React + TypeScript + Node.js) designed for auditability and rapid iteration.

2 Epic

2.1 What is the problem?

In current cryptographic systems, users typically face a binary choice:

1. **Signed Messages:** Authenticity is guaranteed, but privacy is compromised immediately (e.g., PGP).
2. **Anonymous Messages:** Privacy is maintained, but credibility is low, and the author cannot later claim credit or reputation for the content (e.g., Tor-based drops).

There is a lack of infrastructure that supports *conditional anonymity*. Where a user can speak freely now and prove they spoke later, without relying on a trusted third party.

2.2 What is our goals?

- **Objective A:** Anonymous message dispatching where the sender is mathematically hidden among a set of 7 decoys (1-of-8 Ring Signature).
- **Objective B:** Integrity verification by the recipient without identity knowledge.
- **Objective C:** A "Challenge-Response" flow where the sender can prove authorship by unlocking a cryptographic proof using their physical Yubikey.

3 Story

Let's imagine a scenario. Alice wants to send a message to Bob through an online platform anonymously. Bob wants to confirm the message what he received is not tampered with. Additionally, once Bob wants to confirm the identity of the sender, and Alice claims she is the sender, Bob is able to verify her identity.

3.1 How do we provide the solution?

1. **Message Composition:** Alice uses her browser with a connected Yubikey to compose a message. The Yubikey generates a One-Time Key (OTK) by some methods.
2. **Inner Ring Signature:** Alice creates an inner Ring Signature over the message using the OTK and 7 randomly generated fake keys. This signature proves that one of the keys signed the message, but not which one.
3. **Proof Encryption:** The inner signature is encrypted using a symmetric key derived from the PRF call on the Yubikey, ensuring only Alice can decrypt it later.
4. **Outer Ring Signature:** Alice then creates an outer Ring Signature over the message concatenated with the hash of the inner signature, again using the OTK and the same set of fake keys.
5. **Message Transmission:** The outer signature, along with the encrypted inner signature (the Proof), is sent to Bob.
6. **Verification by Bob:** Upon receiving the message, Bob verifies the outer Ring Signature to ensure message integrity without knowing who signed it.

7. **Challenge for Proof:** If Bob wants to verify Alice's identity later, he sends the proof, and the signature to Alice.
8. **Proof Decryption:** Alice uses her Yubikey to derive the decryption key and decrypts the inner signature (the Proof).
9. **Final Verification:** Alice sends the decrypted inner signature to Bob, who verifies it against the original message. If valid, Bob confirms that Alice was indeed the sender.

4 Technical Model

4.1 Security Model

- **Key Derivation:** The OTK is derived with the participation of the Yubikey, ensuring it cannot be predicted or extracted.
- **Anonymity Set:** 1 real signer + 7 random decoys = 8 total keys. Probability of guessing the signer is 1/8 per message (statistically negligible for identification).
- **Forward Secrecy:** Compromise of the Yubikey allows decryption of past Proofs but does not reveal the identity of past messages *unless* the attacker actively performs the decryption and linking.

4.2 Technical Stack Chosen

- **Frontend:** React, TypeScript.
- **Cryptography (Client):** WebAuthn API (PRF extension), Elliptic Curve Library (e.g., `elliptic` or WASM-based Rust bindings) for Ring Signatures.
- **Backend:** Node.js (Express/NestJS).
- **Database:** PostgreSQL(Stores only blobs: Message, Public Keys Ring, Signature, Encrypted Proof).

4.3 Cryptographic Construction

1. **Setup:** Generate One-Time Key (OTK) via Yubikey. Generate 7 Fake Keys ($FK_{1...7}$).
2. **Inner Layer (The Proof):**

$$Sig_{inner} = RingSign(Message, \{OTK, FK_{1...7}\})$$

$$Key_{proof} = KDF(PRF_{Yubikey}(Hash(OTK)))$$

$$Proof = Encrypt_{AES-GCM}(Sig_{inner}, Key_{proof})$$

3. **Outer Layer (The Transmission):**

$$Sig_{outer} = RingSign(Message||Hash(Sig_{inner}), \{OTK, FK_{1...7}\})$$

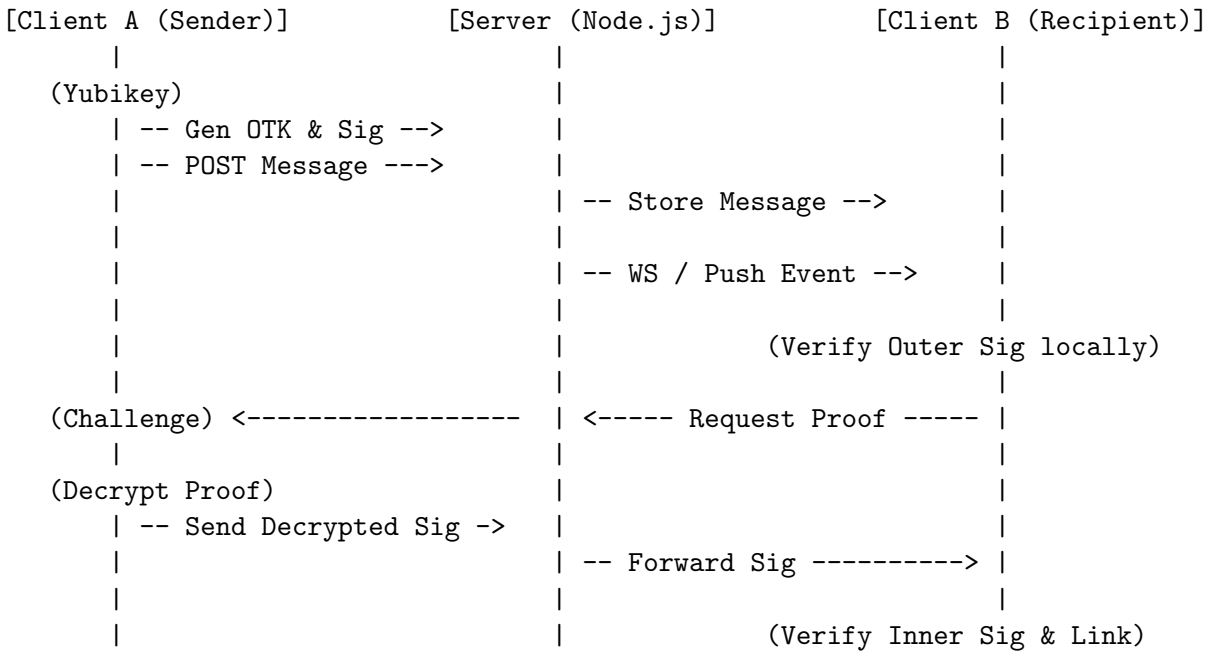
4. **Verification:** Upon decryption, the Linkable Tag of Sig_{inner} must equal the Linkable Tag of Sig_{outer}.

5 System Design

5.1 Architecture Overview

The system follows a **Thick Client, Thin Server** architecture. The server acts purely as a bulletin board and relay, possessing zero knowledge of user identities or private keys.

5.2 Component Diagram



5.3 Data Flow

1. **Registration:** None required for the messaging protocol itself (stateless). Optional username registration maps public handles to WebAuthn Credential IDs for routing.

2. **Signing:** All cryptographic operations occur in the browser memory. The Yubikey is accessed via ‘navigator.credentials.get(prf: ...)‘.

6 Genesis Sprint and Milestones

6.1 Sprint 0: Feasibility & PoC (2 Weeks)

- **Goal:** Validate Yubikey PRF behavior in Chrome/Firefox.
- **Deliverable:** A minimal script that inputs a salt and outputs a consistent AES key using a Yubikey.

6.2 Milestones 1: Core Cryptography Module

- **Goal:** Implement Linkable Ring Signatures in TypeScript/WASM.
- **Deliverable:** Unit tests proving 1-of-8 signing and verification, and tag linking.

6.3 Milestones 2: Backend & Basic UI

- **Goal:** Message transport.
- **Deliverable:** Node.js API to store/retrieve message blobs. React UI to compose messages.

6.4 Milestones 3: Integration & The "Reveal" Flow

- **Goal:** Connect crypto with UI.
- **Deliverable:** End-to-End flow: Alice sends anonymous msg → Bob requests proof → Alice reveals → Bob verifies.